

ShelbyFirst Outcomes Chart

Python source rendered from shelbyfirst_outcomes_chart.py.

Approximate line count: 374

```
#!/usr/bin/env python3
"""
ShelbyFirst: Full Outcomes Improvement Pathway
Run: python shelbyfirst_outcomes_chart.py
Output: src/data/processed/shelbyfirst_outcomes_chart.png
"""

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from matplotlib.lines import Line2D
import numpy as np
import os
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# — Palette —
NAV = '#1b2a6b' # outer bg, header, table col-header
TBL = '#2e4296' # table section separator
WHT = '#ffffff'
EVN = '#eef1fa' # alternating table row
GRY = '#dee3ef' # grid / cell borders
RED = '#c0392b' # "Now" values
GRN = '#1a7a35' # "Yr 3" values
BLU = '#1a4db5' # "Peer" values
DRK = '#1a1a2e' # normal row text
SUB = '#8898cc' # subtitle / footer text

# — Model-driven projections —
# Follows the ShelbyFirst causal pathway (PDF slide 8):
# Programs → direct metric improvements → IGS rises as a consequence
# "ShelbyFirst influences IGS indirectly through better mobility, healthier
# food access, stronger preventive care participation, and deeper community
# engagement." – PDF slide 8
#
# Two-layer model for economic metrics:
# Layer 1 – Ridge Regression (year-only, no IGS) on 2017-2024 panel
# = historical trend / counterfactual without ShelbyFirst
# Layer 2 – ShelbyFirst gap-closing boost layered on top
# boost = (peer_benchmark - baseline) × effectiveness × uptake
#
# Health / access metrics: gap-closing model only (no tract-level panel data)
#
# Program uptake by phase (PDF slide 6 – Implementation Plan):
# Year 1 Pilot = 15% founding households, initial ride utilization
# Year 2 Expand = 35% growing membership, expanded programs
```

```

# Year 3 Scale = 60% full rollout across low-IGS tracts

_dir = os.path.dirname(os.path.abspath(__file__))
_data = os.path.join(_dir, 'data', 'processed')

_panel = pd.read_parquet(os.path.join(_data, 'enrichment_tract_year_panel.parquet'))
_shelby = _panel[_panel['geoid'].astype(str).str.startswith('47157')].copy()
_shelby['income_k'] = _shelby['median_household_income'] / 1000

_YEARS = [2024, 2025, 2026, 2027]
UPTAKE = [0.0, 0.15, 0.35, 0.60] # fraction of 96-tract population engaged

# Peer benchmarks (PDF slide 4 / slide 5)
_PEER = {
    'labor_force': 66.9, 'poverty': 5.5, 'income': 119.0,
    'insured': 94.0, 'no_veh': 4.2, 'dental': 32.1,
    'obesity': 32.5, 'diabetes': 11.3,
}

# Effectiveness: fraction of peer gap closeable per unit of uptake.
# Grounded in ShelbyFirst's three pillars (PDF slide 7):
# Move → affordable rides address transportation friction (PDF slide 5)
# Nourish → community garden targets obesity and diabetes risk factors
# Connect → app + ShelbyCares referrals drive enrollment and care access
EFFECT = {
    'labor_force': 0.22, # Connect: health improvement → employability
    'poverty': 0.15, # Cross-pillar: employment + economic participation
    'income': 0.10, # Very conservative – structural income change is slow
    'insured': 0.40, # Connect: active enrollment navigation + ShelbyCares
    'no_veh': 0.20, # Move: provides alternative mobility, not car ownership
    'dental': 0.35, # Move: transport is the primary dental gap driver
    'obesity': 0.22, # Nourish: garden + physical activity programs
    'diabetes': 0.18, # Nourish: diet + nutrition access improvement
}

def _ridge_trend(df, col, scale=1.0):
    """Year-only Ridge: counterfactual trend without ShelbyFirst intervention."""
    d = df[['year', col]].dropna()
    pipe = Pipeline([('sc', StandardScaler()), ('r', Ridge(alpha=1.0))])
    pipe.fit(d[['year']].values, d[col].values)
    return (pipe.predict(np.array([[yr] for yr in _YEARS])) * scale).tolist()

def _gap_boost(baseline, peer, pillar, direction):
    """ShelbyFirst gap-closing: baseline ± gap × effectiveness × uptake per year."""
    gap = abs(peer - baseline)
    eff = EFFECT[pillar]
    return [baseline + direction * gap * eff * u for u in UPTAKE]

def _econ(df, col, scale, peer, pillar, direction):
    """Layer 1 (Ridge trend) + Layer 2 (ShelbyFirst boost)."""
    trend = _ridge_trend(df, col, scale)
    gap = abs(peer - trend[0])
    eff = EFFECT[pillar]
    result = [trend[0]]
    for i in range(1, 4):
        result.append(trend[i] + direction * gap * eff * UPTAKE[i])

```

```

return result

_lfpr_v = _econ(_shelby, 'lfpr_16p', 100, _PEER['labor_force'], 'labor_force', 1)
_pov_v = _econ(_shelby, 'poverty_rate', 100, _PEER['poverty'], 'poverty', -1)
_inc_v = _econ(_shelby, 'income_k', 1, _PEER['income'], 'income', 1)

# Insured baseline from cluster healthcare data
_cs = pd.read_parquet(os.path.join(_data, 'cluster_shortlist_with_healthcare.parquet'))
_r77 = _cs[_cs['rank'] == 11]
_ins0 = (1 - float(_r77.iloc[0]['uninsured_rate'])) * 100 \
    if len(_r77) and not pd.isna(_r77.iloc[0]['uninsured_rate']) else 82.0

_ins_v = _gap_boost(_ins0, _PEER['insured'], 'insured', 1)
_noveh_v = _gap_boost(13.8, _PEER['no_veh'], 'no_veh', -1)
_dental_v = _gap_boost(55.6, _PEER['dental'], 'dental', -1)
_obes_v = _gap_boost(45.2, _PEER['obesity'], 'obesity', -1)
_diab_v = _gap_boost(21.6, _PEER['diabetes'], 'diabetes', -1)

# IGS: consequence of program improvements, shown as the planning target arc
_IGS = [35.8, 36.8, 38.4, 40.5]

# — Metric definitions —————
# (table_label, direction, [baseline,yr1,yr2,yr3], peer, line_color, legend_label, fmt)
METRICS = [
    ('IGS Score', 1, _IGS, 55.7,
     '#4545c0', f'IGS Score ({_IGS[0]:.1f} → {_IGS[-1]:.1f})',
     '{:.1f}'),
    ('Labor Force %', 1, _lfpr_v, _PEER['labor_force'],
     '#1a8c5a', f'Labor Force Part. ({_lfpr_v[0]:.1f}% → {_lfpr_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('Insured %', 1, _ins_v, _PEER['insured'],
     '#c9a000', f'Insured Rate ({_ins_v[0]:.1f}% → {_ins_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('Med. Income', 1, _inc_v, _PEER['income'],
     '#e06000', f'Med. Income (${_inc_v[0]:.1f}k → ${_inc_v[-1]:.1f}k)',
     '${:.1f}k'),
    ('Poverty %', -1, _pov_v, _PEER['poverty'],
     '#8b1515', f'Poverty Rate ({_pov_v[0]:.1f}% → {_pov_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('No-Veh HH %', -1, _noveh_v, _PEER['no_veh'],
     '#cc1515', f'No-Vehicle HH ({_noveh_v[0]:.1f}% → {_noveh_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('Dental Gap %', -1, _dental_v, _PEER['dental'],
     '#1a8cd4', f'Dental Visit Gap ({_dental_v[0]:.1f}% → {_dental_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('Obesity %', -1, _obes_v, _PEER['obesity'],
     '#8bc820', f'Obesity Rate ({_obes_v[0]:.1f}% → {_obes_v[-1]:.1f}%)',
     '{:.1f}%'),
    ('Diabetes %', -1, _diab_v, _PEER['diabetes'],
     '#30a040', f'Diabetes Rate ({_diab_v[0]:.1f}% → {_diab_v[-1]:.1f}%)',
     '{:.1f}%'),
]

X = [0, 1, 2, 3]
X_LABELS = [
    'Baseline\n2024',
    'Year 1 – Pilot\n2025',
    'Year 2 – Expand\n2026',
    'Year 3 – Scale\n2027',
]

def improvement_index(direction, values):
    """All metrics converted so that higher index = improvement."""
    base = values[0]
    return [v / base * 100 for v in values] if direction == 1 \

```

```

else [base / v * 100 for v in values]

# — Figure —————
# Tall enough so the legend strip (22%-8%) never overlaps the chart (22%-88%).
fig = plt.figure(figsize=(20, 12), dpi=130, facecolor=NAV)

# —————
# HEADER
# —————
fig.text(0.015, 0.965,
        'ShelbyFirst: Full Outcomes Improvement Pathway',
        color=WHT, fontsize=21, fontweight='bold', va='top')
fig.text(0.015, 0.922,
        'Economic + health metrics · ShelbyFirst gap-closing model (Move · Nourish · Connect pillars) · '
        'Uptake: 15% Pilot → 35% Expand → 60% Scale · 96 tracts · pop. 273K',
        color=SUB, fontsize=10, va='top')

# —————
# CHART – left panel [left, bottom, width, height] all in figure fractions
# Now starts at 0.045 (no rotated label needed on the left any more)
# —————
CHART_L, CHART_B, CHART_W, CHART_H = 0.048, 0.23, 0.500, 0.655
ax = fig.add_axes([CHART_L, CHART_B, CHART_W, CHART_H], facecolor=WHT)

ax.set_axisbelow(True)
ax.yaxis.grid(True, color=GRY, linewidth=0.8)
ax.set_xlim(-0.35, 3.35)
ax.set_ylim(94, 147)
ax.set_xticks(X)
ax.set_xticklabels(X_LABELS, fontsize=11.5, color=WHT,
                  linespacing=1.6, fontweight='bold')
ax.set_yticks(range(95, 150, 5))
ax.set_yticklabels([str(v) for v in range(95, 150, 5)],
                  fontsize=11, color=WHT, fontweight='bold')
ax.tick_params(axis='both', length=0)
ax.tick_params(axis='x', pad=12)
ax.tick_params(axis='y', pad=6)

for spine in ['top', 'right', 'left']:
    ax.spines[spine].set_visible(False)
ax.spines['bottom'].set_color(GRY)

# — Horizontal note just above the chart (no title, just the definition line)
ax.text(0.5, 1.008,
        '100 = 2024 baseline · ↑ higher = better for all 9 metrics',
        transform=ax.transAxes, ha='center', va='bottom',
        fontsize=10.5, color='#aabfee', style='italic')

# — "Improvement Index" – rotated Y-axis label in figure space (avoids clipping)
CHART_CX = CHART_L + CHART_W / 2 # figure-fraction x centre of chart
fig.text(0.010, CHART_B + CHART_H / 2,
        'Improvement Index',
        ha='center', va='center', rotation=90,
        fontsize=12.5, color=WHT, fontweight='bold')

# Baseline reference line (dashed) – explanation text moved to subtitle above
ax.axhline(y=100, color='#8090c0', linewidth=1.3, linestyle='--', zorder=2)

```

```

# Plot lines & collect handles
legend_handles = []
for tbl_lbl, direction, vals, peer, color, leg_lbl, fmt in METRICS:
    idx = improvement_index(direction, vals)
    ax.plot(X, idx, color=color, linewidth=2.4, marker='o',
            markersize=6.5, markerfacecolor=WHT, markeredgewidth=1.9, zorder=3)
    legend_handles.append(
        Line2D([0], [0], color=color, linewidth=2.4, marker='o',
               markersize=6, markerfacecolor=WHT, markeredgewidth=1.7, label=leg_lbl)
    )

# -----
# LEGEND - sits in the navy band BELOW THE TABLE (right panel)
# Table left = CHART_L + CHART_W + 0.012 = 0.560, width = 0.425
# Table centre x = 0.560 + 0.425/2 = 0.7725
# -----
TABLE_L = CHART_L + CHART_W + 0.012
TABLE_W = 0.425
TABLE_CX = TABLE_L + TABLE_W / 2 # 0.7725

leg = fig.legend(
    handles=legend_handles,
    loc='lower center',
    bbox_to_anchor=(TABLE_CX, 0.022),
    ncol=2,
    fontsize=10.5,
    frameon=True,
    handlelength=2.2,
    handleheight=1.0,
    columnspacing=1.8,
    labelcolor=WHT,
    borderpad=0.9,
    labelspacing=0.8,
    facecolor='#1e3070',
    edgecolor='#4a60b0',
    framealpha=1.0,
)
for text in leg.get_texts():
    text.set_fontweight('semibold')

# -----
# TABLE - right panel
# -----
ax_t = fig.add_axes([CHART_L + CHART_W + 0.012, CHART_B, 0.425, CHART_H])
ax_t.set_xlim(0, 1)
ax_t.set_ylim(0, 1)
ax_t.axis('off')

COL_W = [0.310, 0.128, 0.115, 0.115, 0.115, 0.127]
COL_X = list(np.cumsum([0.0] + COL_W[:-1]))
COL_H = ['Metric', 'Now', 'Yr 1', 'Yr 2', 'Yr 3', 'Peer']

ROW_H_HDR = 0.082
ROW_H_SEC = 0.050
ROW_H_DATA = 0.073

TABLE_ROWS = [

```

```

(hdr', None),
(sec', '— ECONOMIC METRICS · Ridge year-trend + ShelbyFirst Connect boost —'),
(data', METRICS[0]),
(data', METRICS[1]),
(data', METRICS[2]),
(data', METRICS[3]),
(data', METRICS[4]),
(data', METRICS[5]),
(sec', '— HEALTH METRICS · ShelbyFirst Move / Nourish gap-closing model —'),
(data', METRICS[6]),
(data', METRICS[7]),
(data', METRICS[8]),
]

y = 0.99
data_idx = 0

for rtype, rcontent in TABLE_ROWS:
    rh = {'hdr': ROW_H_HDR, 'sec': ROW_H_SEC, 'data': ROW_H_DATA}[rtype]

    if rtype == 'hdr':
        for i, (cx, cw, ch) in enumerate(zip(COL_X, COL_W, COL_H)):
            ax_t.add_patch(Rectangle((cx, y - rh), cw, rh,
                                     facecolor=NAV, edgecolor='#3a4ea0',
                                     linewidth=0.5, zorder=2))
            xt = cx + 0.013 if i == 0 else cx + cw / 2
            ax_t.text(xt, y - rh / 2, ch,
                      ha='left' if i == 0 else 'center', va='center',
                      fontsize=10, color=WHT, fontweight='bold')

    elif rtype == 'sec':
        ax_t.add_patch(Rectangle((0, y - rh), 1.0, rh,
                                 facecolor=TBL, edgecolor='none', zorder=2))
        ax_t.text(0.013, y - rh / 2, rcontent,
                  ha='left', va='center', fontsize=7.8,
                  color='#aac8ff', style='italic')

    else:
        tbl_lbl, direction, vals, peer, _, _, fmt = rcontent
        bg = EVN if data_idx % 2 == 1 else WHT
        data_idx += 1

        ax_t.add_patch(Rectangle((0, y - rh), 1.0, rh,
                                 facecolor=bg, edgecolor=GRY,
                                 linewidth=0.3, zorder=1))

        for cx in COL_X[1:]:
            ax_t.plot([cx, cx], [y - rh, y], color=GRY, linewidth=0.4, zorder=2)

        ax_t.text(COL_X[0] + 0.013, y - rh / 2, tbl_lbl,
                  ha='left', va='center', fontsize=9.5, color=DRK)

        ax_t.text(COL_X[1] + COL_W[1] / 2, y - rh / 2, fmt.format(vals[0]),
                  ha='center', va='center', fontsize=9.5, color=RED, fontweight='bold')

        ax_t.text(COL_X[2] + COL_W[2] / 2, y - rh / 2, fmt.format(vals[1]),
                  ha='center', va='center', fontsize=9.5, color=DRK)

        ax_t.text(COL_X[3] + COL_W[3] / 2, y - rh / 2, fmt.format(vals[2]),
                  ha='center', va='center', fontsize=9.5, color=DRK)

```

```

ax_t.text(COL_X[4] + COL_W[4] / 2, y - rh / 2, fmt.format(vals[3]),
          ha='center', va='center', fontsize=10, color=GRN, fontweight='bold')

ax_t.text(COL_X[5] + COL_W[5] / 2, y - rh / 2, fmt.format(peer),
          ha='center', va='center', fontsize=9.5, color=BLU, fontweight='bold')

y -= rh

# -----
# FOOTER
# -----
fig.text(
    0.015, 0.010,
    'Model: ShelbyFirst gap-closing – improvement = (peer gap) × program effectiveness × uptake . . '
    'Econ metrics add Ridge year-trend as counterfactual baseline . . '
    'Effectiveness grounded in Move (rides), Nourish (garden), Connect (app/referrals) pillars . PDF slides 6-8',
    color=SUB, fontsize=8, va='bottom',
)

# -----
# SAVE
# -----
_dir = os.path.dirname(os.path.abspath(__file__))
out = os.path.join(_dir, 'data', 'processed', 'shelbyfirst_outcomes_chart.png')
os.makedirs(os.path.dirname(out), exist_ok=True)
plt.savefig(out, dpi=130, bbox_inches='tight', facecolor=NAV, pad_inches=0.2)
print(f'Saved: {out}')
plt.close()

```